Python Fundamentals & PyTorch Tensors

Module 1 of 4 — Python and Machine Learning with PyTorch

Sayan CHAKI

Chercheur
LIRIS, Ecole Centrale Lyon, Univ. Lyon 2, INSA

September 9, 2026

Table of Contents

Where we are

The four modules

1. **Python basics & PyTorch tensors** (today)
2. Functions, classes and `nn.Module`
3. `Dataset`, `DataLoader` and EDA
4. Classic ML: KNN and K-Means, written in PyTorch

Guiding principle

We build everything with **PyTorch tensors**. Scikit-learn is used only to generate datasets and to check metrics, never to hide the algorithm.

What you will be able to do after today

- Read and write basic Python: variables, lists, dictionaries, loops, functions.
- Explain what a **tensor** is and how it differs from a Python list.
- Create tensors, reshape them, and multiply matrices.
- Move a computation from the CPU to the GPU.
- Convert between NumPy arrays and PyTorch tensors safely.

Where we are

Variables and basic types

Python is **dynamically typed**: you never declare a type, but every value has one.

```
n_samples = 150          # int
learning_rate = 0.01     # float
model_name = "resnet18"  # str
is_training = True       # bool
nothing = None           # NoneType

print(type(learning_rate))   # <class 'float'>
print(n_samples / 4)         # 37.5   -> true division
print(n_samples // 4)        # 37     -> floor division
print(f"lr = {learning_rate:.3f}")   # f-string formatting
```

Common beginner trap

/ always returns a float. Use // when you need an integer index.

Lists: ordered, mutable, heterogeneous

```
losses = [2.31, 1.87, 1.45, 1.02]

losses[0]         # 2.31   -> first element
losses[-1]        # 1.02   -> last element
losses[1:3]       # [1.87, 1.45]  -> slice, end excluded

losses.append(0.88)      # add at the end
len(losses)              # 5
min(losses), max(losses) # (0.88, 2.31)

# List comprehension: the most Pythonic loop
squared = [x ** 2 for x in losses]
small   = [x for x in losses if x < 1.5]
```

- Indexing starts at **0**.
- A slice `a:b` includes `a` and excludes `b`.

Dictionaries, tuples and sets

```python
# Dictionary: key -> value, the workhorse of config handling
config = {"lr": 0.01, "epochs": 10, "batch_size": 32}

config["lr"]                  # 0.01
config["optimizer"] = "adam" # insert a new key
config.get("momentum", 0.9)  # default if key is missing

for key, value in config.items():
    print(key, "=", value)

# Tuple: ordered and IMMUTABLE, used everywhere for shapes
shape = (3, 224, 224)

# Set: unordered, unique elements
labels = {0, 1, 2, 2, 1}      # -> {0, 1, 2}
```

Why this matters for PyTorch

Tensor shapes are tuples, model hyperparameters are dictionaries, and a `state_dict` is literally a dictionary of tensors.

Where we are

Conditionals

Python uses **indentation** (4 spaces) instead of braces.

```
accuracy = 0.83

if accuracy > 0.90:
    print("Excellent")
elif accuracy > 0.75:
    print("Good enough for a baseline")
else:
    print("Needs more training")

# Compact ternary form
status = "pass" if accuracy > 0.5 else "fail"
```

Rule

A wrong indentation is a syntax error, not a style problem.

Loops

```python
# for: iterate over any iterable
for epoch in range(3):            # 0, 1, 2
    print("epoch", epoch)

for i, loss in enumerate(losses): # index + value
    print(i, loss)

for name, value in zip(names, values):   # walk two lists together
    print(name, value)

# while: repeat until a condition becomes False
loss, step = 10.0, 0
while loss > 1.0 and step < 100:
    loss *= 0.8
    step += 1

# break / continue
for x in data:
    if x is None:
        continue     # skip this item
    if x > 1e6:
        break        # leave the loop entirely
```

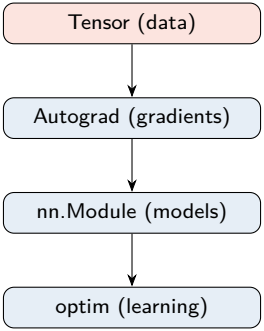
Functions, *args and **kwargs

```python
def train_step(x, y, lr=0.01):
    """Docstring: one line saying what the function does."""
    loss = ((x - y) ** 2).mean()
    return loss                     # returns None if omitted

# *args  -> any number of POSITIONAL arguments, seen as a tuple
def total(*args):
    return sum(args)
total(1, 2, 3)                      # 6

# **kwargs -> any number of KEYWORD arguments, seen as a dict
def build_model(**kwargs):
    print(kwargs)                   # {'depth': 4, 'width': 128}
build_model(depth=4, width=128)

# Both together: the signature you will meet in PyTorch source code
def forward(self, *args, **kwargs):
    ...
```

Where we are

Why PyTorch?

- **Tensors**: n-dimensional arrays with a NumPy-like API.
- **GPU acceleration**: one line moves data to CUDA.
- **Autograd**: gradients computed automatically.
- **Pythonic**: the graph is built as the code runs, so you can use `print` and a debugger inside a model.
- **Ecosystem**: `torchvision`, `torchaudio`, `torch.utils.data`.



The PyTorch stack, bottom layer first.

What is a tensor?

A tensor is simply an array with a number of dimensions, called its **rank**.

Rank	Name	Shape example	Typical use
0	scalar	`()`	a loss value
1	vector	`(10,)`	class scores
2	matrix	`(32, 784)`	a batch of flat images
3	cube	`(3, 224, 224)`	one RGB image
4	batch of images	`(32, 3, 224, 224)`	one training batch

Read a shape from the right

`(32, 3, 224, 224)` means: 32 images, 3 channels, 224 rows, 224 columns.

Creating tensors

```python
import torch

a = torch.tensor([[1., 2.], [3., 4.]])   # from a Python list
z = torch.zeros(3, 4)                    # 3x4 of zeros
o = torch.ones(2, 5)                     # 2x5 of ones
e = torch.eye(3)                         # 3x3 identity
r = torch.rand(2, 3)                     # uniform in [0, 1)
n = torch.randn(2, 3)                    # standard normal
s = torch.arange(0, 10, 2)               # tensor([0, 2, 4, 6, 8])
l = torch.linspace(0, 1, steps=5)        # 5 points from 0 to 1

print(a.shape)   # torch.Size([2, 2])
print(a.dtype)   # torch.float32
print(a.device)  # cpu
print(a.ndim)    # 2
```

Reproducibility

Always call `torch.manual_seed(42)` before any random creation.

Basic operations

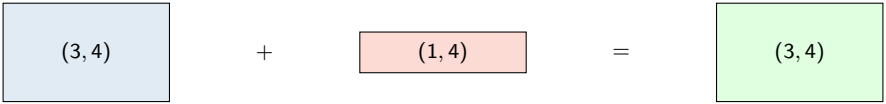
```
x = torch.tensor([[1., 2.], [3., 4.]])
y = torch.tensor([[5., 6.], [7., 8.]])

x + y            # elementwise addition
x * y            # elementwise (Hadamard) product, NOT matrix product
x @ y            # matrix multiplication  (same as torch.matmul)
torch.matmul(x, y)

x.sum(), x.mean(), x.std()      # reductions over all elements
x.sum(dim=0)                    # column sums -> tensor([4., 6.])
x.sum(dim=1)                    # row sums    -> tensor([3., 7.])

x.T                             # transpose
x.reshape(4)                    # flatten to a vector of 4
x.view(1, 4)                    # same data, new shape (contiguous only)
x.unsqueeze(0).shape            # torch.Size([1, 2, 2]) -> add a batch dim
```

The single most frequent bug

`*` is elementwise. Matrix product is `@`.

Broadcasting in one picture

When shapes differ, PyTorch stretches the smaller one automatically.



Rules, compared from the **right**:

1. Dimensions are compatible if they are equal, or if one of them is 1.
2. A missing dimension is treated as 1.
3. The size-1 dimension is repeated, without copying memory.

Everyday use

Subtracting a per-channel mean of shape `(3,1,1)` from an image of shape `(3,224,224)`.

NumPy versus PyTorch

Task	NumPy	PyTorch
Zeros	`np.zeros((2,3))`	`torch.zeros(2,3)`
Shape	`a.shape`	`a.shape`
Reshape	`a.reshape(...)`	`a.reshape/view(...)`
Matrix product	`a @ b`	`a @ b`
Runs on GPU	no	**yes**
Automatic gradients	no	**yes**

```python
import numpy as np
arr = np.array([1., 2., 3.])

t = torch.from_numpy(arr)   # SHARES memory with arr
back = t.numpy()            # SHARES memory with t
safe = torch.tensor(arr)    # COPIES the data
```

Careful

`from_numpy` shares memory. Use `torch.tensor(...)` for a copy.

Devices: CPU and GPU

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(device)                       # cuda  (on a Colab GPU runtime)

x = torch.randn(1000, 1000, device=device)   # created directly on GPU
y = torch.randn(1000, 1000).to(device)       # moved to GPU

z = x @ y                           # runs on the GPU
z_cpu = z.cpu()                     # bring it back for NumPy / matplotlib
```

Rule of thumb

All tensors involved in one operation must live on the **same device**, otherwise PyTorch raises a `RuntimeError`.

In Colab

`Runtime > Change runtime type > Hardware accelerator > GPU`

A first taste of autograd

Gradients are the reason PyTorch exists. We will use this in Module 2.

```
x = torch.tensor(3.0, requires_grad=True)
y = x ** 2 + 2 * x + 1        # y = (x + 1)^2

y.backward()                  # compute dy/dx
print(x.grad)                 # tensor(8.)   because dy/dx = 2x + 2
```

PyTorch records every operation on tensors flagged with `requires_grad=True`, then walks the recorded graph backwards using the chain rule.

Preview

A neural network is just a long chain of tensor operations, so the exact same mechanism trains a 100-layer network.

Where we are

Colab Notebook 1: Python Basics & Tensors

What you will do:

- Run Python syntax drills: lists, dictionaries, loops, functions with `*args`.
- Build a 3×3 matrix and compute dot products and matrix products.
- Compare `*` and `@` on the same pair of tensors.
- Time the same matrix multiplication on CPU and GPU.
- Convert back and forth between NumPy and PyTorch, and observe shared memory.
- **Challenge:** implement a normalisation function using only tensor operations.

File: `Colab_Notebook_1_Basics.ipynb`

Key takeaways

1. Python indentation is syntax; lists, dicts and tuples are your daily tools.
2. A tensor is an n-dimensional array that can live on a GPU and track gradients.
3. `shape`, `dtype` and `device` are the three attributes to check when something breaks.
4. `*` is elementwise, `@` is matrix multiplication.
5. Broadcasting saves you from writing loops.

Next module

Functions in depth, classes, and how `torch.nn.Module` turns a Python class into a neural network.

Questions?

See you in Module 2.